9

Однофазная фильтрация

9.1. Постановка задачи

Рассмотрим задачу фильтрации слабосжимаемой жидкости. Основным дифференциальным уравнением, описывающим течение слабосжимаемой жидкости в пористой среде является уравнение (2.21). Пусть в двумерной области, представленной на рисунке 9.1, через границу Γ_1 подается жидкость с давлением g, на остальной границе области поставим условия непротекания.

Тогда процесс однофазной фильтрации описывается следующим уравнением для давления p:

$$\phi\rho c_t \frac{\partial p}{\partial t} - \nabla \cdot \left(\frac{\rho}{\mu}\mathbf{k}\nabla p\right) = 0, \quad \boldsymbol{x} \in \Omega, \quad 0 < t \leq T. \tag{9.1}$$

где ϕ – пористость среды, $\mathbf{k}$ – тензор проницаемости среды, ρ и μ – плотность и вязкость жидкости, соответственно. Сжимаемость:

$$c_t = c_f + \frac{\phi^0}{\phi} c_R,$$

– общая сжимаемость, c_f и c_R – сжимаемость жидкости и породы, соответственно, ϕ^0 – пористость при давлении p^0. Уравнение (9.1) является нелинейным параболическим уравнением относительно p.

Дополним граничными условиями первого рода на границе Γ_1 и условием

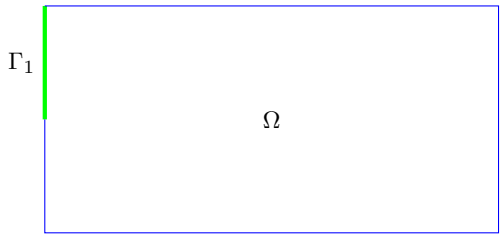


Рисунок 9.1 – Расчетная область Ω

непротекания по остальным границам:

$$\begin{aligned} p(\boldsymbol{x},t) &= g, \quad \boldsymbol{x} \in \Gamma_1, \\ -\frac{\partial p}{\partial n} &= 0, \quad \boldsymbol{x} \in \partial\Omega \backslash \Gamma_1. \end{aligned} \tag{9.2}$$

И добавим начальное условие:

$$p(\boldsymbol{x},0) = p_0, \quad \boldsymbol{x} \in \Omega. \tag{9.3}$$

9.2. Вариационная задача

Для дискретизации по времени воспользуемся методом конечных разностей. Определим равномерную сетку по времени

$$\omega_\tau = \{t_n = n\tau,\ n = 0, 1, ..., N_0,\ \tau = T/N_0\}.$$

Используем чисто неявную схему, при этом для нелинейных коэффициентов будем брать значения с предыдущего временного слоя. Введем обозначение $p^n = p(\boldsymbol{x}, t_n)$ и заменим производную по времени на конечную разность:

$$\frac{\partial p(\boldsymbol{x}, t_n)}{\partial t} \approx \frac{p^{n+1} - p^n}{\tau}$$

Уравнение (9.1) примет следующий вид:

$$B(p^n)\frac{p^{n+1} - p^n}{\tau} - \nabla \cdot \Big(C(p^n)\nabla p^{n+1}\Big) = 0, \quad \boldsymbol{x} \in \Omega,\ n = 0, 1, ... \tag{9.4}$$

где

$$B(p^n) = \phi\rho_0(1 + c_f(p^n - p_0))c_t, \tag{9.5}$$

$$C(p^n) = \rho_0(1 + c_f(p^n - p_0))/\mu \cdot \mathbf{k}. \tag{9.6}$$

Для аппроксимации по пространству используем метод конечных элементов. Запишем задачу в вариационной формулировке. Умножим уравнение (9.4) на тестовую функцию v и проинтегрируем по области Ω с учетом условий (9.2). Используя формулу интегрирования по частям, получим следующую вариационную постановку задачи: найти $p^{n+1} \in V$, такие что:

$$\int\limits_\Omega B(p^n)\frac{p^{n+1} - p^n}{\tau}v dx - \int\limits_\Omega C(p^n)\nabla p^{n+1} \cdot \nabla v dx = \int\limits_\Omega qv dx, \quad \forall v \in \hat{V}, \tag{9.7}$$

где

$$V = \{v \in H^1(\Omega) : v = g,\ \boldsymbol{x} \in \Gamma_1\}, \qquad \hat{V} = \{v \in H^1(\Omega) : v = 0,\ \boldsymbol{x} \in \Gamma_1\}.$$

Определим билинейную и линейную формы следующим образом:

$$a(p^{n+1}, v) = \int\limits_\Omega B(p^n)\frac{p^{n+1}}{\tau}v dx + -\int\limits_\Omega C(p^n)\nabla p^{n+1} \cdot \nabla v dx, \tag{9.8}$$

$$L(v) = \int\limits_{\Omega} qvdx + \int_{\Omega} B(p^n)\frac{p^n}{\tau}vdx. \tag{9.9}$$

После чего получим следующую вариационную формулировку: Найти $p^{n+1} \in V$ такие что

$$a(p^{n+1}, v) = L(v), \quad \forall v \in \hat{V}, n = 0, 1, ... \tag{9.10}$$

9.3. Программная реализация

Представим исходный код вычислительной реализации:

```
from dolfin import *

L = 200.0
H = 100.0

p0 = Point(0, 0)
p1 = Point(L, H)
Nx = 40
Ny = 20

mesh = RectangleMesh(p0, p1, Nx, Ny)

CG = FiniteElement("CG", triangle, 1)
V = FunctionSpace(mesh, CG)

p = TrialFunction(V)
v = TestFunction(V)

def boundary(x):
    return (between(x[1], (H/2, H)) and near(x[0], 0.0))

g  = Constant(18e6)
bc = DirichletBC(V, g, boundary)

k   = Constant(5e-14)
mu  = Constant(0.001)
ct  = Constant(2.0e-9)
cf  = Constant(5.0e-6)
p_0 = Constant(10e6)
fi  = Constant(0.2)
ro0 = Constant(900)

def B(p):
    return ro0 * (1 + cf * (p - p_0)) * fi * ct

def C(p):
    return ro0 * (1 + cf * (p - p_0)) / mu * k

tau = 3600
T = 24 * tau
```